

Telemetry Endpoint Manual

2026-05-04

Telemetry Endpoint

The HTTP POST endpoint that ingests opt-in anonymous scores from Preston-Check scans. Data feeds the annual State of Fintech Security report and the peer-comparison percentiles in the SaaS dashboard.

URLs

There are **two endpoints** that share the same D1 + KV instances:

URL	HOSTED AS	STATUS
<code>https://preston-check-telemetry.preston-check-edge.workers.dev/</code>	Standalone Cloudflare Worker	✅ Production endpoint (the one <code>lib/telemetry.sh</code> defaults to)
<code>https://app.preston-check.com/api/v1/telemetry</code>	Cloudflare Pages Function	🟡 Co-located with customer portal; intermittent issue with <code>onRequestPost</code> dispatch under investigation

Either endpoint accepts the same payload schema and writes to the same backing storage. The runner defaults to the standalone Worker because it's the surface that's been most thoroughly verified end-to-end (rows 1–4 in the dataset all came through it).

Privacy contract

The endpoint validates the documented schema and stores nothing else. Specifically:

- **Accepts** ten documented fields (see Schema below)
- **Hashes** the IP address out of the rate-limit key after the rate-limit decision is made

- **Strips** the IP address from the stored record entirely
- **Never** accepts source code, file paths, file names, or specific check IDs

The privacy implementation is auditable in 30 lines of TypeScript at `workers/telemetry/src/index.ts` (and the equivalent Pages Function at `web/customer/functions/api/v1/telemetry/index.ts`).

Schema

POST `Content-Type: application/json` with this exact payload shape:

```
{
  "version":  "1.7.5",
  "tier":     "free",
  "lang":     "typescript",
  "repo_hash": "sha256-hex-string-64-chars",
  "pass":     268,
  "fail":     7,
  "warn":     14,
  "skip":     5,
  "total":    294,
  "timestamp": "2026-05-04T12:00:00Z"
}
```

Validation rules:

FIELD	TYPE	CONSTRAINT
<code>version</code>	string	<= 32 chars
<code>tier</code>	string	one of <code>free</code> / <code>pro</code> / <code>enterprise</code>
<code>lang</code>	string	<= 32 chars
<code>repo_hash</code>	string	exactly 64 hex chars (<code>/^[a-f0-9]{64}\$/</code>) or literal <code>unhashable</code>
<code>pass</code> , <code>fail</code> , <code>warn</code> , <code>skip</code> , <code>total</code>	number	$0 \leq n \leq 100000$

FIELD	TYPE	CONSTRAINT
<code>timestamp</code>	string	ISO 8601 UTC matching <code>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z</code>

Anything else returns **400 invalid payload**.

Responses

HTTP CODE	MEANING
200	<code>{"ok":true}</code> — payload accepted and queued for storage
400	<code>invalid json</code> or <code>invalid payload</code> — validation failure
405	<code>method not allowed</code> — anything other than POST or OPTIONS
429	<code>rate limit exceeded</code> — IP exceeded 1000 requests/hour

CORS: `Access-Control-Allow-Origin: *` (overridable via the `ALLOW_ORIGIN` env var on the Worker). OPTIONS preflight is handled.

Storage

Each accepted POST writes:

KV (aggregations) — five counters incremented per event, each keyed by date / tier / language. Plus a score histogram bucketed in deciles per language. KV reads serve the score-percentile features in the customer dashboard with sub-50ms latency.

D1 (raw records) — one row inserted into the `scans` table with the full payload + computed score.

Schema:

```
CREATE TABLE scans (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  repo_hash   TEXT NOT NULL,
  version     TEXT NOT NULL,
  tier         TEXT NOT NULL,
  lang        TEXT NOT NULL,
  pass        INTEGER NOT NULL,
  fail        INTEGER NOT NULL,
```

```
warn      INTEGER NOT NULL,
skip      INTEGER NOT NULL,
total     INTEGER NOT NULL,
score     INTEGER NOT NULL,
timestamp TEXT NOT NULL,
inserted_at INTEGER DEFAULT (strftime('%s', 'now'))
);
CREATE INDEX idx_scans_repo_hash ON scans(repo_hash);
```

Retention: 90 days on raw rows. Aggregates kept indefinitely.

Self-hosting

Air-gapped or on-premise customers can stand up their own endpoint and point the runner at it:

```
export PRESTON_TELEMETRY_ENDPOINT="https://telemetry.acme.internal/v1/ingest"
preston-check --telemetry-opt-in
```

The reference implementation (Worker + schema + workflow) lives in `workers/telemetry/` and is Apache 2.0. Roughly 30 minutes to fork + deploy to your own Cloudflare account.

How customers opt in

Three equivalent paths, off by default:

```
preston-check --telemetry-opt-in      # per-run flag
export PRESTON_TELEMETRY=1            # env var
echo 'telemetry: opt_in' >> .preston-check.yml # config file
```

`--airgap` overrides every opt-in.

Querying the dataset

Operator-only — requires Cloudflare API token:

```
export CLOUDFLARE_API_TOKEN="..."
export CLOUDFLARE_ACCOUNT_ID="..."

# Total scan count
wrangler d1 execute preston-check-telemetry --remote --command \
```

```
"SELECT count(*) FROM scans"
```

```
# Daily volume last 7 days
```

```
wrangler d1 execute preston-check-telemetry --remote --command \  
"SELECT substr(timestamp,1,10) AS day, count(*) FROM scans  
WHERE timestamp > datetime('now','-7 days') GROUP BY day"
```

```
# Score distribution by language
```

```
wrangler d1 execute preston-check-telemetry --remote --command \  
"SELECT lang, avg(score) FROM scans GROUP BY lang ORDER BY avg(score) DESC"
```

Source

workers/telemetry/ endpoint)	standalone Worker (production
src/index.ts	TypeScript Worker implementation
schema.sql	D1 schema (already applied)
wrangler.toml	project config + bindings
README.md	setup notes
.github/workflows/telemetry-deploy.yml	CI deploy on push
web/customer/functions/api/v1/telemetry/ endpoint)	Pages Function (alternative
index.ts	

Cross-links

- **Privacy + telemetry overview:** [docs/telemetry.md](#)
- **Worker setup:** [workers/telemetry/README.md](#)
- **Customer Portal manual:** [docs/manuals/customer-portal.md](#)